

Install Required Libraries

```
pip install tensorflow numpy matplotlib
```

Import Libraries

```
import numpy as np
import tensorflow as tf
import time

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, GRU, Dense
```

Task 1: Prepare Flipkart Product Price Sequence

Suppose these are the prices (₹) of a mobile phone over **10 days**.

```
import numpy as np
```

```
prices = np.array([
    50000,
    50200,
    50100,
    50350,
    50500,
    50700,
    50600,
    50900,
    51050,
    51200
], dtype=float)
```

```
print("Original Prices:")
print(prices)
```

Normalize Data

```
prices = prices / 52000
```

Create Sequences

We will use the previous **3 days** to predict the next day's price.

```
X = []
y = []
```

```

sequence_length = 3

for i in range(len(prices) - sequence_length):

    X.append(prices[i:i+sequence_length])

    y.append(prices[i+sequence_length])

X = np.array(X)
y = np.array(y)

# Reshape for LSTM
X = X.reshape((X.shape[0], X.shape[1], 1))

print("Input Shape:", X.shape)
print("Output Shape:", y.shape)

```

Output

Input Shape: (7, 3, 1)

Output Shape: (7,)

Task 2: Build and Train an LSTM Model

```

model = Sequential([

    LSTM(
        32,
        input_shape=(3,1)
    ),

    Dense(1)

])

model.compile(

    optimizer="adam",

    loss="mse"

)

```

```
history = model.fit(  
    X,  
    y,  
    epochs=10,  
    verbose=1  
)
```

Model Summary

```
model.summary()
```

Task 3: Predict the Next Price

```
prediction = model.predict(X[-1].reshape(1,3,1))  
  
predicted_price = prediction[0][0] * 52000  
  
actual_price = y[-1] * 52000  
  
print("Predicted Price : ₹", round(predicted_price,2))  
  
print("Actual Price   : ₹", round(actual_price,2))
```

Example Output

Predicted Price : ₹51085.40

Actual Price : ₹51200.00

Observation

The prediction should be reasonably close to the actual value. Because the dataset is very small and synthetic, exact predictions are not expected.

Task 4: Replace LSTM with GRU

```
start = time.time()  
  
gru_model = Sequential([
```

```

GRU(
    32,
    input_shape=(3,1)
),

Dense(1)

])

gru_model.compile(

    optimizer="adam",

    loss="mse"

)

gru_model.fit(

    X,

    y,

    epochs=10,

    verbose=1

)

end = time.time()

print("GRU Training Time:", round(end-start,2), "seconds")

Prediction

prediction = gru_model.predict(
    X[-1].reshape(1,3,1)
)

print("GRU Predicted Price:")

print(prediction[0][0] * 52000)

```

Comparison

Feature	LSTM	GRU
Gates	3	2
Speed	Slower	Faster
Memory	Better for very long sequences	Good for short to medium sequences
Parameters	More	Fewer
Training Time	Higher	Lower

Observation

- GRU trains faster because it has fewer parameters.
- LSTM often performs better on long and complex sequences.
- On this small dataset, both models usually produce similar predictions.

Task 5: Function to Prepare Sequence Data

```
import numpy as np

def prepare_sequence(data, sequence_length):

    X = []
    y = []

    for i in range(len(data)-sequence_length):

        X.append(
            data[i:i+sequence_length]
        )

        y.append(
            data[i+sequence_length]
        )

    X = np.array(X)
    y = np.array(y)

    X = X.reshape(
        (X.shape[0], X.shape[1], 1)
    )
```

```
return X, y
```

Example (Spotify Song Play Counts)

```
plays = np.array([
    20,
    25,
    28,
    35,
    40,
    42,
    45,
    48
])

X, y = prepare_sequence(
    plays,
    sequence_length=3
)

print("Input Shape:", X.shape)
print("Target Shape:", y.shape)
```

Output

Input Shape: (5, 3, 1)

Target Shape: (5,)

Explanation

The function creates training samples using a sliding window. For each sequence of consecutive values (for example, 3 days of Spotify play counts), it uses those values as the input (X) and the following value as the target (y). Finally, it reshapes the input into the three-dimensional format (samples, time_steps, features) required by LSTM and GRU layers.

Final Summary

Task Implementation

- 1 Prepared a 10-day Flipkart price sequence and reshaped it to (samples, time_steps, features)

Key Learning

LSTM and GRU models require sequential data in a 3D input format.

Task Implementation

Key Learning

- | | | |
|----------|---|--|
| 2 | Built and trained an LSTM model for 10 epochs | LSTMs learn temporal patterns to predict future values from previous observations. |
| 3 | Predicted the next price and compared it with the actual value | Model evaluation is performed by comparing predictions against known targets. |
| 4 | Replaced the LSTM layer with a GRU layer and compared performance | GRUs are typically faster due to fewer parameters, while LSTMs can better capture longer-term dependencies. |
| 5 | Implemented a reusable <code>prepare_sequence()</code> function | A sliding-window approach efficiently converts any time-series data into training sequences suitable for recurrent neural ne |